Constraint Based Problem

SOLUTION

Question 1: Hospital Doctor Scheduling using Backtracking Search

Solution

The problem is solved using the Backtracking Search Algorithm, which systematically assigns shifts to doctors while checking whether all constraints are satisfied. If a constraint is violated, the algorithm backtracks to the previous assignment and tries another possible shift.

Initially, all doctors are unassigned. The algorithm starts by assigning a valid shift to D1, ensuring that D1 is not assigned to the Night shift. Next, it assigns a shift to D2 from the remaining available shifts. Finally, it assigns the last available shift to D3 while verifying that D3 is not assigned to the Morning shift and that D2 is scheduled before D3. If all constraints are satisfied, the algorithm stops and returns the valid schedule; otherwise, it backtracks and explores another assignment.

Algorithm

1. Initialize the list of doctors and available shifts.
2. Assign a shift to the first doctor.
3. Check whether the assignment satisfies all constraints.
4. If valid, proceed to assign the next doctor.
5. If any constraint fails, backtrack and choose another shift.
6. Continue until all doctors are assigned.
7. Display the final valid schedule.

Question 2: Robot Grid Navigation using Breadth-First Search (BFS)

Solution

The robot navigation problem is solved using the Breadth-First Search (BFS) algorithm. BFS explores all neighbouring cells level by level, guaranteeing the shortest path in a grid where each movement has the same cost.

The algorithm starts from the Start position and inserts it into a queue. At every step, the front node is removed from the queue, and all valid neighbouring cells (Up, Down, Left, and Right) are explored. Obstacle cells and previously visited cells are ignored. Every newly discovered cell is added to the queue along with its path information. The process continues until the Goal cell is reached.

Although the problem mentions the Manhattan Distance heuristic, BFS does not use heuristics for node expansion because all edges have equal cost. The heuristic is only calculated for reference.

Algorithm

1. Create a queue and insert the Start node.
2. Mark the Start node as visited.
3. Remove the first node from the queue.
4. Explore all valid neighbouring cells.
5. Ignore obstacle cells and visited cells.
6. Add newly discovered cells to the queue.
7. Repeat until the Goal node is found.
8. Display the shortest path and total cost.

Question 3: Autonomous Rescue Robot using Uniform Cost Search

Solution

The rescue robot operates in a partially observable environment where different paths have different traversal costs. Therefore, the Uniform Cost Search (UCS) algorithm is used because it always expands the node with the minimum cumulative cost.

The robot starts at the initial position and senses only the neighbouring cells. It updates its knowledge of the environment whenever new obstacles or risky zones are discovered. Safe movements have a cost of 1, while entering risky zones adds an extra cost of 2. A priority queue is used to select the next node with the lowest accumulated cost. The search continues until the survivor is reached.

Since the environment changes during exploration, the robot behaves as an Online Search Agent, continuously updating its internal map and replanning whenever new information becomes available.

Algorithm

1. Start from the initial position.
2. Observe adjacent cells.
3. Detect obstacles and risky zones.
4. Insert neighbouring cells into the priority queue based on cumulative cost.
5. Expand the node with the lowest cost.
6. Update the environment knowledge dynamically.
7. Continue until the Goal is reached.
8. Display the safest minimum-cost path.

Question 4: Fraud Detection in Financial Systems

Solution

The fraud detection system analyses each financial transaction using predefined fraud detection rules. Every transaction is evaluated based on factors such as transaction amount, location, transaction frequency, and transaction time.

The system first accepts transaction details from the user. It then checks each transaction against the fraud detection conditions. If one or more suspicious conditions are satisfied, the transaction is classified as Fraudulent; otherwise, it is classified as Legitimate. Finally, the system displays the classification result for every transaction.

Algorithm

1. Read transaction details.
2. Analyse the transaction using fraud detection rules.
3. Check for suspicious conditions.
4. If suspicious conditions are met, classify the transaction as Fraudulent.

5. Otherwise, classify it as Legitimate.
6. Display the result.
7. Repeat for all transactions.

Question 5: Convex Hull using Brute Force Algorithm

Solution

The Convex Hull problem is solved using the Brute Force Convex Hull Algorithm. The algorithm checks every pair of points to determine whether the line joining them forms an edge of the convex hull.

For each pair of points, the algorithm verifies whether all remaining points lie on the same side of the line. If every point lies on one side (or on the line itself), then that line segment is part of the convex hull. This process is repeated for every possible pair of points until all boundary edges are identified.

Although the brute force approach is simple to implement, it becomes inefficient for large datasets because every pair of points must be examined.

Algorithm

1. Read all input points.
2. Select every possible pair of points.
3. Check the position of all remaining points relative to the selected line.
4. If all points lie on one side, include the line as a convex hull edge.
5. Repeat for all point pairs.
6. Collect all boundary points.
7. Display the Convex Hull.
8. Analyse the time complexity and scalability.